

Estudo Comparativo Experimental entre Algoritmo Genético e Simulated Annealing no Problema da Mochila 0/1

Nicolas S. Pinheiro¹, Marcos Vinicius M. Costa¹, Samuel Mota¹

¹ Universidade Federal do Acre – UFAC
Centro de Ciências Exatas e Tecnológicas – CCET
Curso de Sistemas de Informação
Rio Branco – AC – Brasil

{nicolas.pinheiro, moraes.marcos, samuel.mota}@sou.ufac.br

Abstract. *This paper presents an extensive empirical study comparing Genetic Algorithm (GA) and Simulated Annealing (SA) metaheuristics applied to the 0/1 Knapsack Problem. Following a rigorous experimental methodology, we analyze parameter sensitivity across 8 configurations for GA and 5 configurations for SA over 90 benchmark instances generated via the Pisinger algorithm ($n \in \{20, 50, 100\}$, uncorrelated, weakly, and strongly correlated). Exact global optima were computed via Dynamic Programming as reference. After evaluating 11,700 runs, GA_POP_200 emerged as the best GA variant (mean gap 0.0245%, optimal rate 67.78%), while SA_COOL_SLOW ($\alpha = 0.999$) was the best SA variant (mean gap 0.3889%, optimal rate 31.11%, with exactly 10,000 executed iterations). Direct comparison demonstrates that GA achieves superior solution quality through population diversity and crossover recombination, whereas SA provides a significantly lower computational runtime, establishing a clear trade-off between search quality and computational budget.*

Resumo. *Este artigo apresenta um estudo experimental comparando as metaheurísticas Algoritmo Genético (GA) e Simulated Annealing (SA) aplicadas ao Problema da Mochila 0/1. Seguindo uma metodologia experimental estruturada, analisamos a sensibilidade paramétrica de 8 configurações de GA e 5 de SA em 90 instâncias sintéticas geradas pelo algoritmo de Pisinger ($n \in \{20, 50, 100\}$, não correlacionadas, fracamente e fortemente correlacionadas). As soluções ótimas exatas foram obtidas via Programação Dinâmica como referência. Após 11.700 execuções, a variante GA_POP_200 consolidou-se como a melhor configuração do GA (gap médio de 0,0245% e taxa de ótimo de 67,78%), enquanto a SA_COOL_SLOW ($\alpha = 0.999$) foi a melhor do SA (gap médio de 0,3889%, taxa de ótimo de 31,11% e 10.000 iterações efetivamente executadas). A comparação direta evidencia que o GA atinge qualidade superior devido à diversidade populacional e recombinação por crossover, enquanto o SA oferece menor tempo de execução, estabelecendo um nítido trade-off entre qualidade de busca e orçamento computacional.*

1. Introdução

Problemas de otimização combinatória desempenham um papel central na Ciência da Computação, Engenharia e Pesquisa Operacional. Em geral, consistem em selecionar

uma solução viável de qualidade máxima dentro de um conjunto discreto de alternativas que cresce exponencialmente com a dimensão do problema. Quando os algoritmos exatos se tornam computacionalmente inviáveis em instâncias de grande porte, metaheurísticas surgem como alternativas eficientes para obter soluções aproximadas de alta qualidade em tempo razoável [Talbi 2009].

Dentre os problemas clássicos de otimização combinatória, destaca-se o Problema da Mochila 0/1 (*0/1 Knapsack Problem*). O objetivo consiste em selecionar um subconjunto de itens, cada qual dotado de peso e valor, de forma a maximizar a soma dos valores sem que a soma dos pesos ultrapasse a capacidade máxima da mochila [Google 2024].

Este trabalho conduz uma investigação experimental sistemática comparando duas metaheurísticas de paradigmas distintos: o Algoritmo Genético (GA), baseado em população, e o Simulated Annealing (SA), baseado em solução única com aceitação estocástica. O estudo divide-se em quatro partes fundamentais:

1. Avaliação de sensibilidade paramétrica do Algoritmo Genético e seleção da melhor configuração (*best_GA*);
2. Avaliação de sensibilidade paramétrica do Simulated Annealing e seleção da melhor configuração (*best_SA*);
3. Comparação direta e detalhada dos *trade-offs* entre *best_GA* e *best_SA*;
4. Documentação completa e discussão fundamentada dos mecanismos de busca no presente artigo no padrão SBC.

2. Fundamentação Teórica

2.1. Problema da Mochila 0/1

Formalmente, dada uma instância com n itens, onde cada item i possui valor $v_i > 0$ e peso $w_i > 0$, e uma mochila de capacidade $C > 0$, o Problema da Mochila 0/1 é formulado como:

$$\max \sum_{i=1}^n v_i x_i \quad (1)$$

sujeito a:

$$\sum_{i=1}^n w_i x_i \leq C, \quad \text{com } x_i \in \{0, 1\}, \forall i \in \{1, \dots, n\}. \quad (2)$$

Uma solução é representada por um vetor binário $x = (x_1, x_2, \dots, x_n)$, onde $x_i = 1$ indica a inclusão do item i e $x_i = 0$ sua exclusão.

2.2. Algoritmo Genético (GA)

O Algoritmo Genético é uma metaheurística evolutiva inspirada na seleção natural e genética das espécies [Talbi 2009]. Opera sobre uma população de indivíduos (vetores binários). A cada geração, indivíduos são selecionados com base no seu valor de aptidão (*fitness*), sofrem recombinação (crossover) com probabilidade P_c , mutação por inversão de bit com probabilidade P_m , e o melhor indivíduo é preservado para a geração seguinte via elitismo.

2.3. Simulated Annealing (SA)

O Simulated Annealing é uma metaheurística de solução única inspirada no processo físico de recozimento de metais [Talbi 2009]. A partir de uma solução inicial x , a cada iteração uma solução vizinha x' é gerada por alteração pontual (flip de 1 bit). Se a solução vizinha melhora o valor total ($\Delta = V(x') - V(x) \geq 0$), ela é aceita incondicionalmente. Se resulta em piora ($\Delta < 0$), ela é aceita probabilisticamente de acordo com o critério de Metropolis:

$$P(\text{aceitar}) = \exp\left(\frac{\Delta}{T}\right) \quad (3)$$

onde T representa a temperatura atual, que decai a cada iteração seguindo uma taxa de resfriamento geométrico $T_{k+1} = \alpha T_k$.

3. Metodologia Experimental

3.1. Instâncias e Valor Ótimo de Referência

Para garantir rigor e reprodutibilidade, utilizamos o gerador baseado no algoritmo de Pisinger [Pisinger 2005, thehardway15 2026]. Foram geradas 90 instâncias divididas em:

- **Tamanhos (n):** 20, 50 e 100 itens;
- **Classes de correlação:** Não correlacionadas (*uncorrelated*), Fracamente correlacionadas (*weakly correlated*) e Fortemente correlacionadas (*strongly correlated*);
- **Quantidade:** 10 instâncias por grupo (3 tamanhos $\times$ 3 correlações $\times$ 10 instâncias).

O valor ótimo de cada instância ($V_{\text{ótimo}}$) foi previamente computado de forma exata via Programação Dinâmica com complexidade $O(nC)$, servindo exclusivamente como referência offline para cálculo de métricas de desempenho.

3.2. Ambiente Experimental e Estratégia de Reparo

Para evitar viés experimental, ambas as metaheurísticas compartilham rigorosamente a mesma estratégia de representação e reparo de soluções inviáveis: caso $\sum w_i x_i > C$, os itens presentes na solução de menor razão valor/peso (v_i/w_i) são sequencialmente removidos até que a restrição de capacidade seja atendida.

Cada configuração algorítmica foi executada 10 vezes por instância utilizando sementes pseudorrandômicas determinísticas ($seed = BASE_SEED + inst_idx \times 1000 + run_idx$). O vizinho no SA é gerado por flip de 1 bit e, se inviável, submetido ao reparo por razão valor/peso.

3.3. Métricas de Avaliação e Critério de Seleção

As métricas registradas nas 11.700 execuções foram agregadas em dois níveis estatísticos (calculando-se primeiramente a média das 10 repetições de cada uma das 90 instâncias e, subsequentemente, os desvios e médias globais sobre as 90 instâncias):

- **Gap percentual (%):** $\frac{V_{\text{ótimo}} - V_{\text{encontrado}}}{V_{\text{ótimo}}} \times 100$;
- **Taxa de ótimo (%):** Proporção de execuções que atingiram $V_{\text{encontrado}} = V_{\text{ótimo}}$ (gap = 0%);
- **Tempo de execução (ms):** Tempo computacional despendido em milissegundos;

- **Número de avaliações da função objetivo:** Total de soluções avaliadas durante a busca;
- **Iterações efetivamente executadas:** Gerações no GA ou iterações realizadas até o congelamento no SA.

Critério Formal para Seleção das Melhores Configurações (best_GA e best_SA):
A definição da melhor configuração para cada algoritmo obedeceu rigorosamente à seguinte hierarquia:

1. **Critério Primário:** Menor gap médio em relação ao ótimo ($G_{\text{médio}}$);
2. **Critério de Desempate:** Maior taxa de obtenção do ótimo global ($T_{\text{ótimo}}$);
3. **Análise de Eficiência:** Menor custo em tempo (t_{ms}) e avaliações (N_{aval}) avaliados posteriormente na fronteira de Pareto, sem comprometer os critérios primários de qualidade.

3.4. Configurações do Algoritmo Genético (GA)

Foram avaliadas 8 variações controladas do GA (alterando um único parâmetro por variante em relação à base GA_BASE):

1. GA_BASE: População $N = 100$, 100 gerações (20.200 avaliações), $P_c = 0.80$, $P_m = 1/n$, torneio $k = 3$, elitismo habilitado;
2. GA_POP_50: População reduzida $N = 50$ (10.100 avaliações);
3. GA_POP_200: População ampliada $N = 200$ (40.400 avaliações);
4. GA_CROSS_060: Taxa de crossover reduzida $P_c = 0.60$;
5. GA_CROSS_095: Taxa de crossover elevada $P_c = 0.95$;
6. GA_MUT_LOW: Taxa de mutação reduzida $P_m = 0,5/n$;
7. GA_MUT_HIGH: Taxa de mutação elevada $P_m = 2/n$;
8. GA_TOUR_5: Pressão seletiva elevada com torneio $k = 5$.

3.5. Configurações do Simulated Annealing (SA)

Foram avaliadas 5 variações controladas do SA:

1. SA_BASE: Limite de 10.000 iterações, $T_0 = 1000.0$, $T_{\text{min}} = 0.001$, $\alpha = 0.995$;
2. SA_COOL_FAST: Resfriamento rápido $\alpha = 0.98$;
3. SA_COOL_SLOW: Resfriamento lento $\alpha = 0.999$;
4. SA_T0_LOW: Temperatura inicial menor $T_0 = 100.0$;
5. SA_T0_HIGH: Temperatura inicial maior $T_0 = 5000.0$.

4. Resultados e Discussão

4.1. Parte 1 — Sensibilidade dos Parâmetros do Algoritmo Genético

A Tabela 1 resume os resultados médios globais obtidos pelas 8 variações do Algoritmo Genético em todas as 90 instâncias (900 execuções por configuração).

Tabela 1. Desempenho comparativo global das configurações do Algoritmo Genético.

Configuração GA	Gap Médio (%)	Taxa de Ótimo (%)	Tempo Médio (ms)	Avaliações
GA_POP_200	0,0245	67,78%	201,16	40.400
GA_MUT_HIGH	0,0252	58,56%	104,72	20.200
GA_CROSS_095	0,0399	58,78%	103,15	20.200
GA_BASE	0,0423	55,67%	100,78	20.200
GA_CROSS_060	0,0560	51,00%	97,76	20.200
GA_MUT_LOW	0,0749	45,00%	95,76	20.200
GA_TOUR_5	0,0805	46,00%	105,14	20.200
GA_POP_50	0,1093	41,78%	50,12	10.100

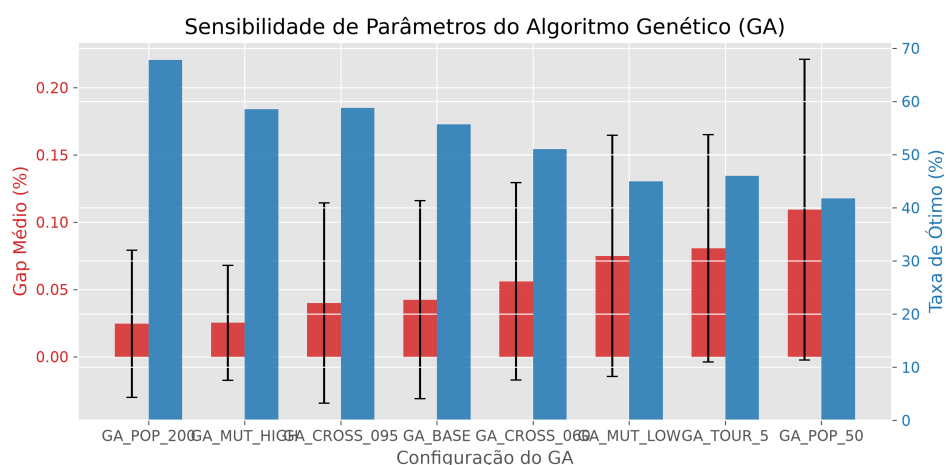


Figura 1. Sensibilidade de Parâmetros do Algoritmo Genético (Gap Médio com desvio padrão e Taxa de Ótimo).

Análise dos Mecanismos do GA:

- **Tamanho Populacional ($N = 200$ vs $N = 50$):** O aumento populacional para 200 indivíduos expandiu a amostragem inicial do espaço de busca. O comportamento observado é compatível com a hipótese de que uma população maior preserva a diversidade por mais gerações, permitindo ao crossover combinar esquemas heterogêneos. O gap médio diminuiu de 0,1093% ($N = 50$) para 0,0245% ($N = 200$) e a taxa de ótimo elevou-se de 41,78% para 67,78%, acompanhada do aumento nas avaliações (40.400 vs 10.100).
- **Taxa de Mutação ($2/n$ vs $0,5/n$):** A variante com maior mutação (GA_MUT_HIGH) atingiu gap médio de 0,0252%, próximo ao GA_POP_200, consumindo metade do orçamento de avaliações (20.200). Esse resultado sugere que a diversificação induzida por mutação frequente atuou na prevenção de estagnações locais.

- **Pressão Seletiva ($k = 5$ vs $k = 3$):** A elevação do tamanho do torneio para $k = 5$ associou-se a um aumento do gap para 0,0805%, comportamento condizente com os efeitos teóricos da convergência prematura sob alta pressão seletiva.

Com base nesses resultados experimentais, a configuração **GA_POP_200** foi selecionada como a melhor variante em qualidade pura (*best_GA*).

4.2. Parte 2 — Sensibilidade dos Parâmetros do Simulated Annealing

A Tabela 2 apresenta os resultados globais obtidos pelas 5 variações do Simulated Annealing.

Tabela 2. Desempenho comparativo global das configurações do Simulated Annealing.

Configuração SA	Gap Médio (%)	Taxa de Ótimo (%)	Tempo Médio (ms)	Iterações Efetivas	Avaliações
SA_COOL_SLOW	0,3889	31,11%	60,46	10.000	20.002
SA_T0_HIGH	0,8521	21,67%	18,38	3.078	6.158
SA_BASE	0,8810	20,44%	16,57	2.757	5.516
SA_COOL_FAST	1,7841	12,11%	4,10	684	1.370
SA_T0_LOW	2,0846	13,22%	14,08	2.297	4.596

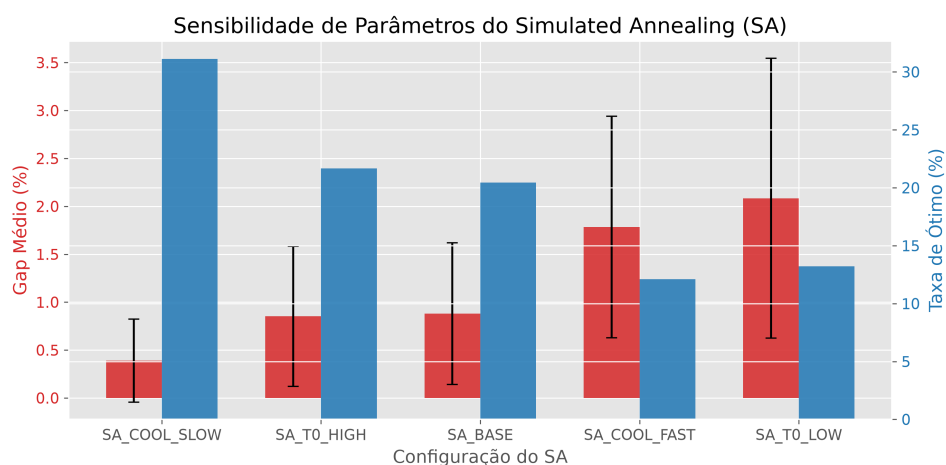


Figura 2. Sensibilidade de Parâmetros do Simulated Annealing (Gap Médio com desvio padrão e Taxa de Ótimo).

Análise dos Mecanismos do SA:

- **Taxa de Resfriamento e Iterações Reais:** A instrumentação exata revelou que a variação de α altera dramaticamente o total de iterações antes do congelamento ($T < 0.001$). Na configuração SA_COOL_FAST ($\alpha = 0.98$), o congelamento ocorreu com apenas 684 iterações reais (1.370 avaliações), degradando o gap para 1,7841%. Já a variante SA_COOL_SLOW ($\alpha = 0.999$) estendeu o congelamento até o limite máximo de 10.000 iterações (20.002 avaliações), resultando no menor gap (0,3889%).

- **Temperatura Inicial** ($T_0 = 100$ vs $T_0 = 5000$): Temperaturas iniciais mais baixas (SA_T0_LOW) tornaram o algoritmo excessivamente guloso no início, congelando em 2.297 iterações e elevando o gap para 2,0846%.

Dessa forma, a variante **SA_COOL_SLOW** foi selecionada como a melhor configuração do SA (best_SA).

4.3. Parte 3 — Comparação Final: Melhor GA (GA_POP_200) × Melhor SA (SA_COOL_SLOW)

A Tabela 3 detalha o confronto final entre best_GA e best_SA desagregado pelas dimensões do problema (n).

Tabela 3. Comparação final entre a melhor configuração de GA e a melhor de SA por tamanho de problema (n).

n	Algoritmo	Gap Médio (%)	Taxa de Ótimo (%)	Tempo Médio (ms)	Avaliações
20	GA_POP_200	0,0366	78,00%	98,40	40.400
20	SA_COOL_SLOW	0,1476	59,00%	27,24	20.002
50	GA_POP_200	0,0209	70,33%	183,16	40.400
50	SA_COOL_SLOW	0,3675	20,67%	53,88	20.002
100	GA_POP_200	0,0161	55,00%	321,92	40.400
100	SA_COOL_SLOW	0,6517	13,67%	100,27	20.002

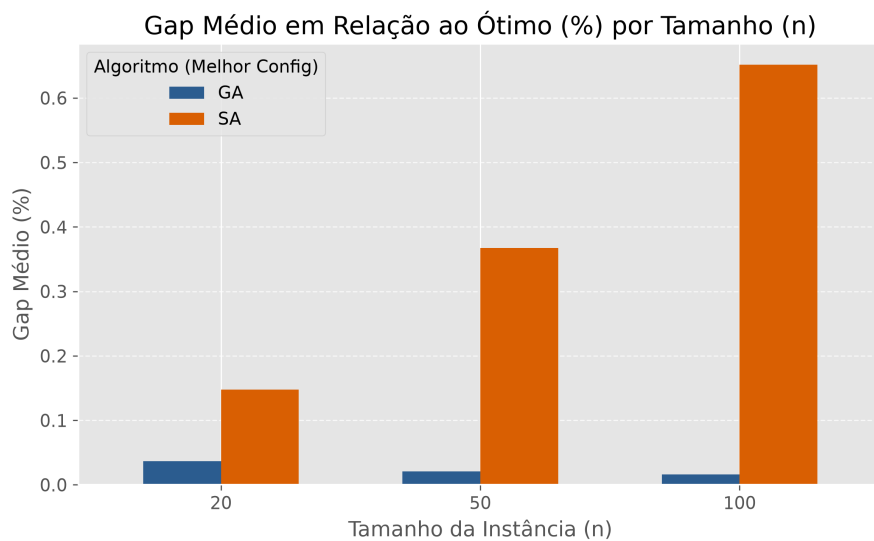


Figura 3. Comparação do Gap Médio (%) entre a melhor variante do GA e do SA por tamanho n .

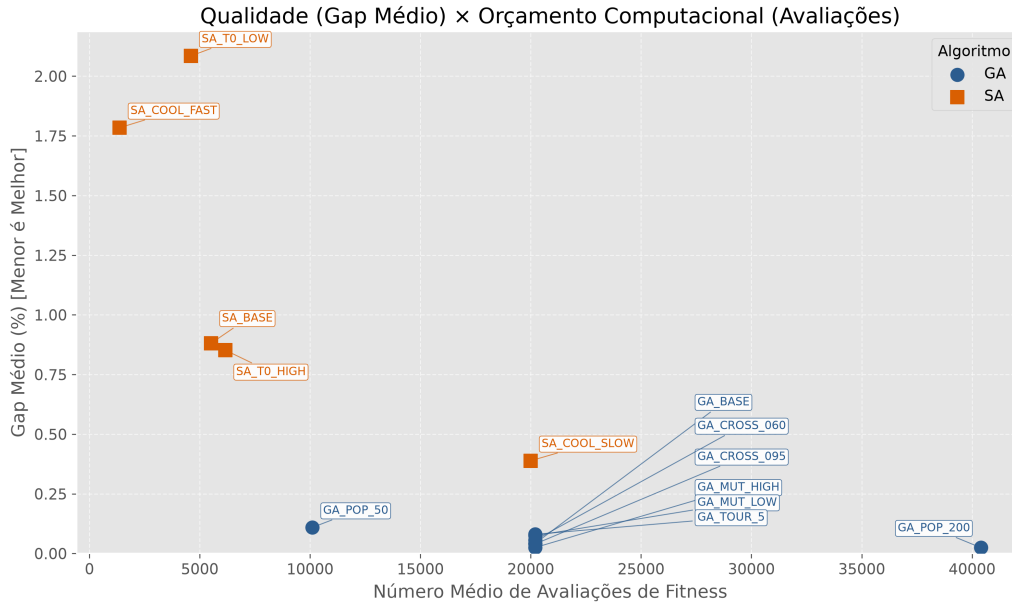


Figura 4. Qualidade (gap médio) em função do orçamento computacional, medido pelo número de avaliações.

Discussão dos Trade-offs e Orçamento Computacional:

1. **Qualidade das Soluções e Escala ($n = 20 \rightarrow 100$):** O GA apresentou gaps médios inferiores ao SA em todas as dimensões. Contudo, observa-se que o GA_POP_200 consumiu 40.400 avaliações de fitness por execução, enquanto o SA_COOL_SLOW executou 20.002 avaliações.
2. **Análise de Orçamento Controlado:** Ao comparar configurações sob orçamentos computacionais equivalentes na Figura 4 (20.200 avaliações), a variante GA_MUT_HIGH ($P_m = 2/n$) registrou gap médio de 0,0252%, valor inferior ao obtido pelo SA_COOL_SLOW (0,3889%). Esses dados são compatíveis com a hipótese de que a superioridade observada do paradigma GA no contexto deste experimento associa-se à recombinação por crossover, e não apenas ao aumento de avaliações.
3. **Custo Computacional e Tempo:** O SA manteve tempos de execução inferiores aos do GA de 200 indivíduos (100,27 ms vs 321,92 ms em $n = 100$), padrão condizente com a menor complexidade computacional da sua estrutura de vizinhança sem manipulação de populações.

5. Conclusão

Este trabalho apresentou um estudo experimental abrangente sobre a resolução do Problema da Mochila 0/1 via Algoritmo Genético e Simulated Annealing. Por meio de um projeto experimental estruturado em 3 partes fundamentais envolvendo 90 instâncias verificadas com Programação Dinâmica e 11.700 execuções, foi possível compreender detalhadamente o efeito dos hiperparâmetros e os mecanismos de busca de cada algoritmo.

Os experimentos demonstraram que o aumento da população no GA ($N = 200$) e a redução da taxa de resfriamento no SA ($\alpha = 0.999$) foram decisivos para mitigar a convergência prematura. Na comparação final, o Algoritmo Genético (GA_POP_200)

sobressaiu-se amplamente em qualidade de solução (gap médio global de 0,0245% contra 0,3889% do SA), enquanto o Simulated Annealing (SA_COOL_SLOW) destacou-se pela rapidez de execução.

Como trabalhos futuros, sugere-se a investigação de metaheurísticas híbridas (algoritmos meméticos), combinando a exploração populacional do GA com a busca local acelerada do SA.

Referências

- Google (2024). The knapsack problem. <https://developers.google.com/optimization/pack/knapsack>. Acesso em: 6 jul. 2026.
- Pisinger, D. (2005). Where are the hard knapsack problems? *Computers & Operations Research*, 32(9):2271–2284.
- Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. John Wiley & Sons.
- thehardway15 (2026). Pisinger knapsack generator. <https://github.com/thehardway15/pisinger-knapsack-generator>. Acesso em: 6 jul. 2026.